Experiment 1 – Library Management System

Aim:

To develop a web-based Library Management System for managing books, users, book issue/return details, fines, and study materials.

Algorithm:

1. Create the library website with user and librarian modules.
2. Provide registration and login for users and librarian.
3. Allow librarian to add, update, delete, issue, and return books.
4. Allow users to search books and view borrowing history.
5. Calculate and display fines for overdue books.
6. Provide searching and downloading of study materials.
7. Display user profile and library information.
8. Stop the system.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Library Management System</title>
  <style>
    body {
      font-family: Arial;
      background: #eef2f7;
```

```
    margin: 0;
}
header {
    background: #234e70;
    color: white;
    text-align: center;
    padding: 20px;
}
.container {
    width: 85%;
    margin: 25px auto;
    background: white;
    padding: 20px;
    border-radius: 10px;
}
input, button {
    padding: 10px;
    margin: 5px;
}
button {
    background: #234e70;
    color: white;
    border: none;
    cursor: pointer;
}
table {
```

```
            width: 100%;
            border-collapse: collapse;
            margin-top: 15px;
        }
        th, td {
            border: 1px solid #ccc;
            padding: 10px;
            text-align: left;
        }
        th {
            background: #dce8f2;
        }
        h2 {
            color: #234e70;
        }
    </style>
</head>
<body>

<header>
    <h1>📚 Library Management System</h1>
    <p>Search, issue and return books easily</p>
</header>

<div class="container">
    <h2>Search Books</h2>
```

```
<input type="text" id="search"
       placeholder="Enter book title or author"
       oninput="displayBooks()">

<h2>Book Catalogue</h2>
<table>
   <thead>
      <tr>
         <th>Book ID</th>
         <th>Book Name</th>
         <th>Author</th>
         <th>Status</th>
         <th>Action</th>
      </tr>
   </thead>
   <tbody id="bookList"></tbody>
</table>

<h2>My Borrowing History</h2>
<table>
   <thead>
      <tr>
         <th>Book</th>
         <th>Issue Date</th>
         <th>Expected Return</th>
         <th>Action</th>
```

```
        </tr>
      </thead>
      <tbody id="history"></tbody>
  </table>

  <h2>Fine Calculator</h2>
  <label>Days overdue:</label>
  <input type="number" id="days" min="0" value="0">
  <button onclick="calculateFine()">Calculate Fine</button>
  <p id="fine"></p>

  <h2>Librarian: Add Book</h2>
  <input id="newTitle" placeholder="Book title">
  <input id="newAuthor" placeholder="Author">
  <button onclick="addBook()">Add Book</button>
  <p id="message"></p>
</div>

<script>
let books = [
  {id: 1, title: "Web Technology", author: "R. Sharma", available: true},
  {id: 2, title: "Java Programming", author: "Herbert Schildt", available:
true},
  {id: 3, title: "Data Structures", author: "Mark Allen", available: true}
];
```

```
let history = [];
let nextId = 4;

function displayBooks() {
  let search = document.getElementById("search").value.toLowerCase();

  let filtered = books.filter(book =>
    book.title.toLowerCase().includes(search) ||
    book.author.toLowerCase().includes(search)
  );

  document.getElementById("bookList").innerHTML =
filtered.map(book => `
    <tr>
      <td>${book.id}</td>
      <td>${book.title}</td>
      <td>${book.author}</td>
      <td>${book.available ? "Available" : "Issued"}</td>
      <td>
        ${book.available
          ? `<button onclick="issueBook(${book.id})">Issue</button>`
          : `<button
onclick="returnBook(${book.id})">Return</button>`}
        <button onclick="deleteBook(${book.id})">Delete</button>
      </td>
    </tr>
  `).join("");
```

```
}

function issueBook(id) {
    let book = books.find(b => b.id === id);
    if (!book || !book.available) return;

    book.available = false;
    let issueDate = new Date();
    let dueDate = new Date();
    dueDate.setDate(dueDate.getDate() + 14);

    history.push({
        id: id,
        title: book.title,
        issue: issueDate.toLocaleDateString(),
        due: dueDate.toLocaleDateString(),
        active: true
    });

    displayBooks();
    displayHistory();
}

function returnBook(id) {
    let book = books.find(b => b.id === id);
    if (!book) return;
```

```
    book.available = true;
    let record = [...history].reverse().find(h => h.id === id && h.active);
    if (record) record.active = false;

    displayBooks();
    displayHistory();
}

function displayHistory() {
    document.getElementById("history").innerHTML = history.map(h => `
        <tr>
            <td>${h.title}</td>
            <td>${h.issue}</td>
            <td>${h.due}</td>
            <td>${h.active ? "Borrowed" : "Returned"}</td>
        </tr>
    `).join("");
}

function addBook() {
    let title = document.getElementById("newTitle").value.trim();
    let author = document.getElementById("newAuthor").value.trim();

    if (!title || !author) {
        document.getElementById("message").textContent =
```

```
            "Enter both book title and author.";
        return;
    }

    books.push({
        id: nextId++,
        title: title,
        author: author,
        available: true
    });

    document.getElementById("newTitle").value = "";
    document.getElementById("newAuthor").value = "";
    document.getElementById("message").textContent =
        "Book added successfully!";

    displayBooks();
}

function deleteBook(id) {
    let book = books.find(b => b.id === id);
    if (!book) return;

    if (!book.available) {
        alert("Return the book before deleting it.");
        return;
```

```
    }

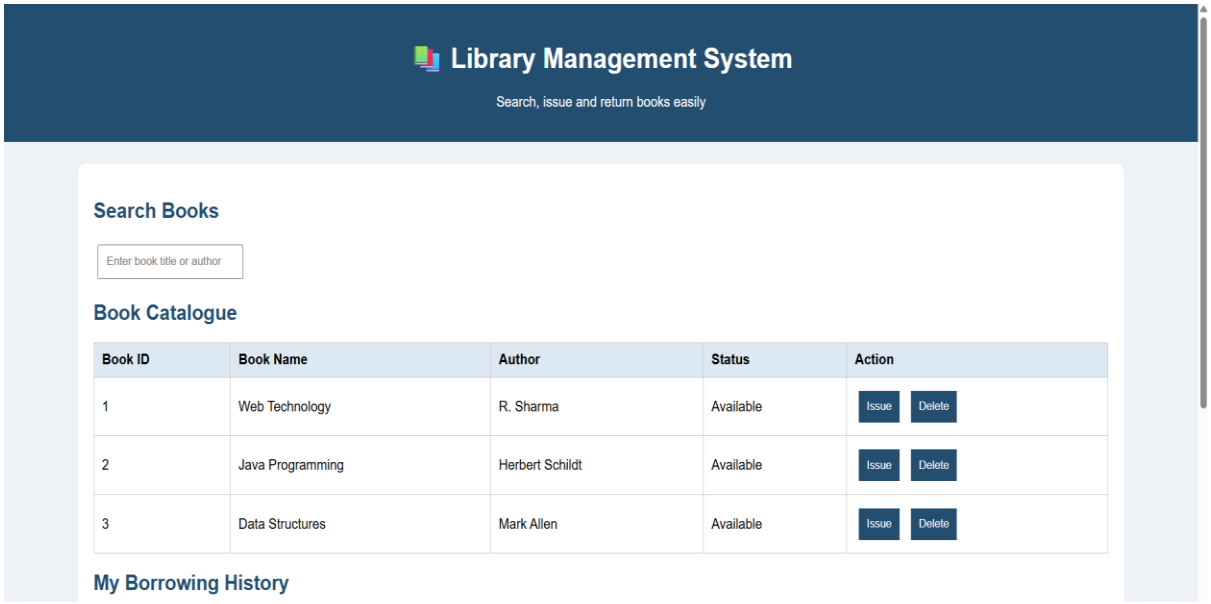
    books = books.filter(b => b.id !== id);
    displayBooks();
}

function calculateFine() {
    let days = Math.max(0,
Number(document.getElementById("days").value) || 0);
    let fine = days * 5;
    document.getElementById("fine").textContent =
        "Fine amount: ₹" + fine;
}

displayBooks();
displayHistory();
</script>

</body>
</html>
```

Output:



Result: The Library Management System was successfully developed to manage books, users, borrowing records, returns, fines, and study materials.